04-实战：纯手工打造和运行一个Servlet

作为Java程序员，我们可能已经习惯了使用IDE和Web框架进行开发，IDE帮我们做了编译、打包的工作，而Spring框架在背后帮我们实现了Servlet接口，并把Servlet注册到了Web容器，这样我们可能很少有机会接触到一些底层本质的东西，比如怎么开发一个Servlet？如何编译Servlet？如何在Web容器中跑起来？

今天我们就抛弃IDE、拒绝框架，自己纯手工编写一个Servlet，并在Tomcat中运行起来。一方面进一步加深对Servlet的理解；另一方面，还可以熟悉一下Tomcat的基本功能使用。

主要的步骤有：

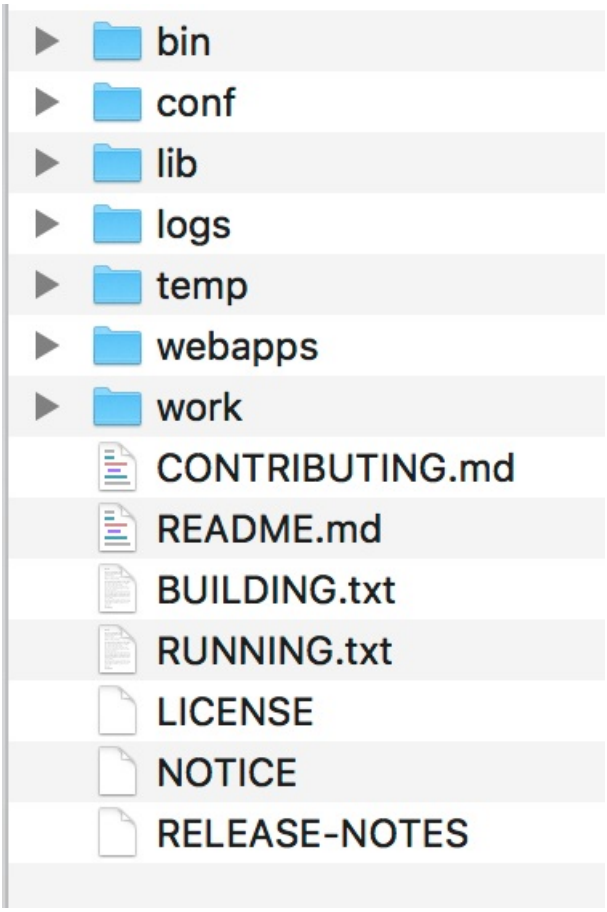
1.下载并安装Tomcat。
2.编写一个继承HTTPServlet的Java类。
3.将Java类文件编译成Class文件。
4.建立Web应用的目录结构，并配置web.xml。
5.部署Web应用。
6.启动Tomcat。
7.浏览器访问验证结果。
8.查看Tomcat日志。

下面你可以跟我一起一步步操作来完成整个过程。Servlet 3.0规范支持用注解的方式来部署Servlet，不需要在web.xml里配置，最后我会演示怎么用注解的方式来部署Servlet。

1. 下载并安装Tomcat

最新版本的Tomcat可以直接在官网上下载，根据你的操作系统下载相应的版本，这里我使用的是Mac系统，下载完成后直接解压，解压后的目录结构如下。

仅供个人学习 请勿传播 用户标记 mannix小童鞋



下面简单介绍一下这些目录：

/bin：存放Windows或Linux平台上启动和关闭Tomcat的脚本文件。
/conf：存放Tomcat的各种全局配置文件，其中最重要的是server.xml。
/lib：存放Tomcat以及所有Web应用都可以访问的JAR文件。
/logs：存放Tomcat执行时产生的日志文件。
/work：存放JSP编译后产生的Class文件。
/webapps：Tomcat的Web应用目录，默认情况下把Web应用放在这个目录下。

2. 编写一个继承HTTPServlet的Java类

我在专栏上一期提到，javax.servlet包提供了实现Servlet接口的GenericServlet抽象类。这是一个比较方便的类，可以通过扩展它来创建Servlet。但是大多数的Servlet都在HTTP环境中处理请求，因此Serve规范还提供了HttpServlet来扩展GenericServlet并且加入了HTTP特性。我们通过继承HttpServlet类来实现自己的Servlet只需要重写两个方法：doGet和doPost。

因此今天我们创建一个Java类去继承HTTPServlet类，并重写doGet和doPost方法。首先新建一个名为MyServlet.java的文件，敲入下面这些代码：

```
import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
```

```
public class MyServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        System.out.println("MyServlet 在处理get（）请求...");
        PrintWriter out = response.getWriter();
        response.setContentType("text/html;charset=utf-8");
        out.println("<strong>My Servlet!</strong><br>");
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        System.out.println("MyServlet 在处理post（）请求...");
        PrintWriter out = response.getWriter();
        response.setContentType("text/html;charset=utf-8");
        out.println("<strong>My Servlet!</strong><br>");
    }

}
```

这个Servlet完成的功能很简单，分别在doGet和doPost方法体里返回一段简单的HTML。

3. 将Java文件编译成Class文件

下一步我们需要把MyServlet.java文件编译成Class文件。你需要先安装JDK，这里我使用的是JDK 10。接着你需要把Tomcat lib目录下的servlet-api.jar拷贝到当前目录下，这是因为servlet-api.jar中定义了Servlet接口，而我们的Servlet类实现了Servlet接口，因此编译Servlet类需要这个JAR包。接着我们执行编译命令：

```
javac -cp ./servlet-api.jar MyServlet.java
```

编译成功后，你会在当前目录下找到一个叫MyServlet.class的文件。

4. 建立Web应用的目录结构

我们在上一期学到，Servlet是放到Web应用部署到Tomcat的，而Web应用具有一定的目录结构，所有我们按照要求建立Web应用文件夹，名字叫MyWebApp，然后在这个目录下建立子文件夹，像下面这样：

```
MyWebApp/WEB-INF/web.xml

MyWebApp/WEB-INF/classes/MyServlet.class
```

然后在web.xml中配置Servlet，内容如下：

仅供个人学习 请勿传播 用户标记 mannix小查理

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
  http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
  version="4.0"
  metadata-complete="true">

    <description> Servlet Example. </description>
    <display-name> MyServlet Example </display-name>
    <request-character-encoding>UTF-8</request-character-encoding>

    <servlet>
      <servlet-name>myServlet</servlet-name>
      <servlet-class>MyServlet</servlet-class>
    </servlet>

    <servlet-mapping>
      <servlet-name>myServlet</servlet-name>
      <url-pattern>/myservlet</url-pattern>
    </servlet-mapping>

</web-app>
```

你可以看到在web.xml配置了Servlet的名字和具体的类，以及这个Servlet对应的URL路径。请你注意，**servlet和servlet-mapping这两个标签里的servlet-name要保持一致。**

5. 部署Web应用

Tomcat应用的部署非常简单，将这个目录MyWebApp拷贝到Tomcat的安装目录下的webapps目录即可。

6. 启动Tomcat

找到Tomcat安装目录下的bin目录，根据操作系统的不同，执行相应的启动脚本。如果是Windows系统，执行`startup.bat.`；如果是Linux系统，则执行`startup.sh`。

7. 浏览访问验证结果

在浏览器里访问这个URL：`http://localhost:8080/MyWebApp/myservlet`，你会看到：

```
My Servlet!
```

这里需要注意，访问URL路径中的MyWebApp是Web应用的名字，myservlet是在web.xml里配置的Servlet的路径。

8. 查看Tomcat日志

打开Tomcat的日志目录，也就是Tomcat安装目录下的logs目录。Tomcat的日志信息分为两类：一是运行日

志，它主要记录运行过程中的一些信息，尤其是一些异常错误日志信息；二是访问日志，它记录访问的时间、IP地址、访问的路径等相关信息。

这里简要介绍各个文件的含义。

- `catalina.***.log`

主要是记录Tomcat启动过程的信息，在这个文件可以看到启动的JVM参数以及操作系统等日志信息。

- `catalina.out`

catalina.out是Tomcat的标准输出（stdout）和标准错误（stderr），这是在Tomcat的启动脚本里指定的，如果没有修改的话stdout和stderr会重定向到这里。所以在这个文件里可以看到我们在MyServlet.java程序里打印出来的信息：

> MyServlet在处理get()请求…

- `localhost.**.log`

主要记录Web应用在初始化过程中遇到的未处理的异常，会被Tomcat捕获而输出这个日志文件。

- `localhost_access_log.**.txt`

存放访问Tomcat的请求日志，包括IP地址以及请求的路径、时间、请求协议以及状态码等信息。

- `manager.***.log/host-manager.***.log`

存放Tomcat自带的manager项目的日志信息。

用注解的方式部署Servlet

为了演示用注解的方式来部署Servlet，我们首先修改Java代码，给Servlet类加上**@WebServlet**注解，修改后的代码如下。

```
import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/myAnnotationServlet")
public class AnnotationServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
```

仅供个人学习 请勿传播 用户标记 mannix小查理

```
    System.out.println("AnnotationServlet 在处理get () 请求...");
        PrintWriter out = response.getWriter();
        response.setContentType("text/html; charset=utf-8");
        out.println("<strong>Annotation Servlet!</strong><br>");

    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        System.out.println("AnnotationServlet 在处理post () 请求...");
        PrintWriter out = response.getWriter();
        response.setContentType("text/html; charset=utf-8");
        out.println("<strong>Annotation Servlet!</strong><br>");

    }

}
```

这段代码里最关键的就是这个注解，它表明两层意思：第一层意思是AnnotationServlet这个Java类是一个Servlet，第二层意思是这个Servlet对应的URL路径是myAnnotationServlet。

```
@WebServlet("/myAnnotationServlet")
```

创建好Java类以后，同样经过编译，并放到MyWebApp的class目录下。这里要注意的是，你**需要删除原来的web.xml**，因为我们不需要web.xml来配置Servlet了。然后重启Tomcat，接下来我们验证一下这个新的AnnotationServlet有没有部署成功。在浏览器里输入：`http://localhost:8080/MyWebApp/myAnnotationServlet`，得到结果：

```
Annotation Servlet!
```

这说明我们的AnnotationServlet部署成功了。可以通过注解完成web.xml所有的配置功能，包括Servlet初始化参数以及配置Filter和Listener等。

本期精华

通过今天的学习和实践，相信你掌握了如何通过扩展HttpServlet来实现自己的Servlet，知道了如何编译Servlet、如何通过web.xml来部署Servlet，同时还练习了如何启动Tomcat、如何查看Tomcat的各种日志，并且还掌握了如何通过注解的方式来部署Servlet。我相信通过专栏前面文章的学习加上今天的练习实践，一定会加深你对Servlet工作原理的理解。之所以我设置今天的实战练习，是希望你知道IDE和Web框架在背后为我们做了哪些事情，这对于我们排查问题非常重要，因为只有我们明白了IDE和框架在背后做的事情，一旦出现问题的时候，我们才能判断它们做得对不对，否则可能开发环境里的一个小问题就会折腾我们半天。

课后思考

我在Servlet类里同时实现了doGet方法和doPost方法，从浏览器的网址访问默认访问的是doGet方法，今天的课后思考题是如何访问这个doPost方法。

不知道今天的内容你消化得如何？如果还有疑问，请大胆的在留言区提问，也欢迎你把你的课后思考和心得记录下来，与我和其他同学一起讨论。如果你觉得今天有所收获，欢迎你把它分享给你的朋友。



精选留言：

- Monday 2019-05-18 10:20:39
  1、postman
  2、curl 命令发送post
  3、用HttpClient发送

  周六早上坚持打卡，本章节绝大多数知识以前有接触过，只有@WebServlet注解是新知识，现在业务开发一般都是写SpringMVC容器中的Controller来代替类似文中的Servlet类。

  问题：基于Spring+SpringMVC+Mybais的框架搭建的项目，平常开发的都是写Controller与Service、DAO。
  1、请问Servlet容器只管理DispatchServlet这一个Servlet吗？
  2、有什么可视化工具可以直接查看各种容器中管理的对象吗？

  谢谢！ [5赞]

  作者回复2019-05-18 19:43:01
  1. 你可以向Servlet容器注册多个Servlet。
  2. 你看这个有没有帮助
  https://github.com/spring-projects/spring-framework/issues/14296

- 风翱 2019-05-18 07:47:23
  可以利用工具，例如postman。 也可以编写代码，利用http的post方法去调用。 或者像楼上所说的不管

仅供个人学习 请勿传播 用户标记 mannix小童鞋

通过get还是post都通知转发到doPost中。 [4赞]

- allean 2019-05-18 16:05:54
IDE和框架诞生之初是为了让程序员从繁琐的底层配置中抽离出来专注于业务开发，然而大多数人在享受IDE和框架带来的便捷时，也成了温水里的青蛙，对于实现原理认识模糊，渐渐沦落为一个CRUD，这不是一件好事，啊 ~ 幡然醒悟 [3赞]

- 今夜秋风和 2019-05-18 11:10:43
老师，验证的时候默认增加了 super.doGet(req, resp);在http1.1写一下不能工作，查看httpServlet 源码里面 对协议做了限制，http 1.1 协议默认不支持。这个为什么是这样设计的呢？
源代码:
String protocol = req.getProtocol();
String msg = lStrings.getString("http.method_get_not_supported");
if (protocol.endsWith("1.1")) {
resp.sendError(HttpServletResponse.SC_METHOD_NOT_ALLOWED, msg);
} else {
resp.sendError(HttpServletResponse.SC_BAD_REQUEST, msg);
}
第二个是如果是那个注解访问的，可以不用删除web.xml，把web.xml里面的url-pattern 改成注解同样的路由，也可以支持；如果web.xml 路由自定义一个的话，测试发现自定义的会有404，是不是注解的路由优先级会更高呢？
3.如果把web.xml删除，servlet容器启动的时候是不是会自动扫描注解类，将它注册到容器中? [2赞]

  作者回复2019-05-19 09:16:42
  super.doGet(req, resp); 调的是HttpServlet的doGet方法，但是这个doGet需要你去实现的。

  HttpServlet的service方法会调doXXX方法，并且HttpServlet里的各种doXXX方法的默认实现都是直接返回错误。

  为什么HttpServlet要这样设计呢？这是因为它需要做一个限制：程序员要么重写HttpServlet的service方法，要么重写HttpServlet的doXXX方法。

  web.xml和注解可以同时工作，你需要把web.xml中的metadata-complete="true"设置成false。

  Tomcat启动时会扫描注解，同时记下Servlet的名字和映射路径，如果你设置了延迟记载Servlet，通过浏览器访问时Tomcat才会加载和实例化Servlet。

- youknowmjj 2019-05-19 19:01:15
@Amanda 不知道怎么直接回复你，我跟你一样也是遇到乱码的问题。我也设置了response.setCharacterEncoding(utf8)，在getWriter之前，但依然是一样的乱码。
原因在与javac编译生成的class文件是用的gbk的编码。换句话说，你生成的class源文件就已经是中文乱码了。
所以在javac的时候加上 -encoding UTF-8就好了。
ps:我是Windows的环境。 [1赞]

- Amanda 2019-05-18 19:44:32
老师，实践中发现个问题：虽然response.setContentType("text/html;charset=utf-8")，但是out.println中有输出中文还是乱码的 [1赞]

  作者回复2019-05-19 18:35:53

调下顺序，像下面这样：
response.setContentType("text/html; charset=utf-8");
PrintWriter out = response.getWriter();

getWrite的源码如下：
------
public PrintWriter getWriter()
throws IOException {

if (usingOutputStream) {
throw new IllegalStateException
(sm.getString("coyoteResponse.getWriter.ise"));
}

if (ENFORCE_ENCODING_IN_GET_WRITER) {
/*
* If the response's character encoding has not been specified as
* described in <code>getCharacterEncoding</code> (i.e., the method
* just returns the default value <code>ISO-8859-1</code>),
* <code>getWriter</code> updates it to <code>ISO-8859-1</code>
* (with the effect that a subsequent call to getContentType() will
* include a charset=ISO-8859-1 component which will also be
* reflected in the Content-Type response header, thereby satisfying
* the Servlet spec requirement that containers must communicate the
* character encoding used for the servlet response's writer to the
* client).
*/
setCharacterEncoding(getCharacterEncoding());
}

usingWriter = true;
outputBuffer.checkConverter();
if (writer == null) {
writer = new CoyoteWriter(outputBuffer);
}
return writer;
}

你看注释里它说：如果调这个方法之前没有指定Response的字符编码，就用默认的ISO-8859-1，ISO-8859-1不包括中文字符。

- darren 2019-05-18 15:18:36
发现xml与注解不能同时起作用，那在用xml方式的老项目中就没办法使用注解的方式了吗？ [1赞]

作者回复2019-05-19 17:50:45
web.xml 和注解可以同时工作的。

例子里的web.xml和注解不能同时工作的原因是web.xml中的配置metadata-complete="true"， 你需要把它设置成metadata-complete="false"。

metadata-complete为true的意思是，告诉Tomcat不要去扫描Servlet注解了。

- KL3 2019-05-18 00:37:14
  把业务逻辑写在dopost里，然后doget方法调用dopost方法 [1赞]

- 张济增 2019-05-20 09:09:38
  看着前几章跟 head first servlet jsp差不多，先来个基础

- feitian 2019-05-20 07:51:39
  既然是纯手工，就应该把servlet那套完整的写出来，不应该再用tomcat容器而应该手写实现tomcat的核心代码。最核心的应该是类似HttpServlet功能的实现，把这个从最初的servlet接口实现了才算讲透，不然还是有点和稀泥的感觉。我觉得应该按照这种思路讲会更好，请参考我写的mytomcat，https://github.com/feifa168/mytomcat

- 郑童文 2019-05-20 01:08:31
  请问老师： 我们在servlet的实现类中import的是javax.servlet.http.HttpServlet 请问为什么需要Tomcat的servlet-api.jar呢？难道javax.servlet.http.HttpServlet这个类不是jdk中的吗？谢谢！

- Geek_bde3dc 2019-05-18 23:01:09
  最简单的办法就是在doGet里调用doPost😁

- Geek_0db340 2019-05-18 19:24:38
  表单提交method=post 就可以啦

- 林 2019-05-18 18:44:54
  curl 指定method为post，发起请求

- nsn 2019-05-18 15:34:21
  1.可以通过设置Request Method: POST
  2.可以使用工具，但本质还是设置Method
  3.在doGet()方法中调用doPost()
  但方法3总感觉是在作弊。。。不是正经答案

- M 2019-05-18 10:54:15
  最笨的方法，在doGet 方法调用doPost方法。

- 逆流的鱼 2019-05-18 10:42:21
  用Tomcat实现servlet，总感觉哪里不对

  作者回复2019-05-18 19:43:53
  😑

- Amanda 2019-05-18 09:35:44
  通过网址访问发送的是GET请求。那么发送POST请求就会调用到doPost方法。不过一般会在doGet中调用doPost()方法。

- WL 2019-05-18 08:57:21
  请问一下老师我这边在logs的文件夹下没有看到catalina.out， 是哪里没配置吗？

作者回复2019-05-18 09:29:06
Mac这种“UNIX-like”的OS，默认不需要配置的。

我猜你可能用的Windows系统，你可以研究一下在Windows下怎么把标准输出和标准出错重定向一个文件里去。

- cloud 2019-05-18 08:56:55
是怎么找到有写的注解类？遍历加载所有jar包中的类吗

作者回复2019-05-18 09:34:35
对的，Tomcat在启动时会扫描jar包~